

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2
“Модульное тестирование”

Выполнил:
Студент группы ИУ5-33Б
Якубова Д. Р.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Листинг программы

main.py

```
from data import get_universities, get_faculties, get_faculty_universities
from queries import get_university_faculties, get_faculty_count,
get_faculties_ending_with_iy

def main():
    universities = get_universities()
    faculties = get_faculties()
    faculty_universities = get_faculty_universities()

    print("Запрос 1: Список университетов и их факультетов (сортировка по
университету):")
    result1 = get_university_faculties(universities, faculties)
    for university_name, faculties_list in result1.items():
        print(f'{university_name}: {' '.join(faculties_list)}')

    print("\nЗапрос 2: Количество факультетов по университетам (сортировка
по количеству):")
    result2 = get_faculty_count(universities, faculty_universities)
    for university_name, count in result2:
        print(f'{university_name}: {count}')

    print("\nЗапрос 3: Факультеты на 'ий' и их университеты:")
    result3 = get_faculties_ending_with_iy(faculties, universities,
faculty_universities)
    for faculty_name, university_list in result3:
        print(f'{faculty_name}: {' '.join(university_list)}')

if __name__ == "__main__":
    main()
```

model.py

```
class University:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class Faculty:
    def __init__(self, id, name, university_id, student_count):
        self.id = id
```

```
self.name = name
self.university_id = university_id
self.student_count = student_count
```

```
class FacultyUniversity:
    def __init__(self, faculty_id, university_id):
        self.faculty_id = faculty_id
        self.university_id = university_id
```

data.py

```
from models import University, Faculty, FacultyUniversity
```

```
def get_universities():
    return [
        University(1, "МГТУ"),
        University(2, "СПбГУ"),
        University(3, "МГУ")
    ]
```

```
def get_faculties():
    return [
        Faculty(1, "Филологический", 1, 300),
        Faculty(2, "Математический", 1, 250),
        Faculty(3, "Экономический", 2, 200),
        Faculty(4, "Юридический", 3, 150),
        Faculty(5, "Биологический", 3, 180)
    ]
```

```
def get_faculty_universities():
    return [
        FacultyUniversity(1, 1),
```

```
FacultyUniversity(2, 1),
FacultyUniversity(3, 2),
FacultyUniversity(4, 3),
FacultyUniversity(5, 3),
FacultyUniversity(1, 2),
FacultyUniversity(3, 3)
]
```

queries.py

```
def get_university_faculties(universities, faculties):
    """Запрос 1: Список университетов и их факультетов"""
    university_faculties = {}
    for faculty in faculties:
        university_id = faculty.university_id
        university = next(u for u in universities if u.id == university_id)
        if university.name not in university_faculties:
            university_faculties[university.name] = []
        university_faculties[university.name].append(faculty.name)

    result = {}
    for university_name in sorted(university_faculties.keys()):
        result[university_name] = sorted(university_faculties[university_name])

    return result

def get_faculty_count(universities, faculty_universities):
    """Запрос 2: Количество факультетов по университетам"""
    faculty_count = {}
```

```

for f in faculty_universities:
    faculty_count[f.university_id] = faculty_count.get(f.university_id, 0) + 1

result = []
for uid, count in faculty_count.items():
    university = next(u for u in universities if u.id == uid)
    result.append((university.name, count))

return sorted(result, key=lambda x: x[1])

def get_faculties_ending_with_iy(faculties, universities, faculty_universities):
    """Запрос 3: Факультеты на 'ий' и их университеты"""
    result = []

    for faculty in faculties:
        if faculty.name.endswith("ий"):
            uni_list = []
            for fu in faculty_universities:
                if fu.faculty_id == faculty.id:
                    university = next(u for u in universities if u.id == fu.university_id)
                    uni_list.append(university.name)

            result.append((faculty.name, uni_list))

    return result

```

simple_tests.py

```
from models import University, Faculty, FacultyUniversity

from queries import get_university_faculties, get_faculty_count,
get_faculties_ending_with_iy
```

```
def run_all_tests():
```

```
    """Запуск всех тестов"""
```

```
    print("=" * 50)
```

```
    print("Запуск модульных тестов")
```

```
    print("=" * 50)
```

```
    universities = [
```

```
        University(1, "Тестовый Университет 1"),
```

```
        University(2, "Тестовый Университет 2")
```

```
    ]
```

```
    faculties = [
```

```
        Faculty(1, "Тестовый факультет", 1, 50),
```

```
        Faculty(2, "Другой факультет", 2, 60)
```

```
    ]
```

```
    faculty_universities = [
```

```
        FacultyUniversity(1, 1),
```

```
        FacultyUniversity(2, 2)
```

```
    ]
```

```
    # Тест 1
```

```
    print("\nТест 1: get_university_faculties")
```

```
    result1 = get_university_faculties(universities, faculties)
```

```
    print(f"Результат: {result1}")
```

```
assert len(result1) == 2, "Должно быть 2 университета"
print("Тест 1 пройден")

# Тест 2
print("\nТест 2: get_faculty_count")
result2 = get_faculty_count(universities, faculty_universities)
print(f"Результат: {result2}")
assert len(result2) == 2, "Должно быть 2 записи"
print("Тест 2 пройден")

# Тест 3
print("\nТест 3: get_faculties_ending_with_iy")
faculties_with_iy = faculties + [Faculty(3, "Химический", 1, 70)]
result3 = get_faculties_ending_with_iy(faculties_with_iy, universities,
faculty_universities)
print(f"Результат: {result3}")
assert any("Химический" in item for item in result3), "Должен найти
'Химический'"
print("Тест 3 пройден")

print("\n" + "=" * 50)
print("Все тесты успешно пройдены!")
print("=" * 50)

if __name__ == "__main__":
    run_all_tests()
```

test_queries.py

```
import pytest

from models import University, Faculty, FacultyUniversity

from queries import get_university_faculties, get_faculty_count,
get_faculties_ending_with_iy

class TestUniversityQueries:

    """Тесты для функций запросов"""

    def setup_method(self):

        """Подготовка тестовых данных"""

        self.universities = [

            University(1, "Университет А"),

            University(2, "Университет Б")

        ]

        self.faculties = [

            Faculty(1, "Филологический", 1, 100),

            Faculty(2, "Математический", 1, 150),

            Faculty(3, "Экономический", 2, 120)

        ]

        self.faculty_universities = [

            FacultyUniversity(1, 1),

            FacultyUniversity(2, 1),

            FacultyUniversity(3, 2),

            FacultyUniversity(1, 2)

        ]
```



```

def test_get_university_faculties(self):
    """Тест 1: Список университетов и их факультетов"""
    result = get_university_faculties(self.universities, self.faculties)

    assert len(result) == 2

    assert "Университет А" in result
    assert result["Университет А"] == ["Математический",
"Филологический"]

    assert "Университет Б" in result
    assert result["Университет Б"] == ["Экономический"]

    keys = list(result.keys())
    assert keys == ["Университет А", "Университет Б"]

def test_get_faculty_count(self):
    """Тест 2: Количество факультетов по университетам"""
    result = get_faculty_count(self.universities, self.faculty_universities)

    assert len(result) == 2

    university_names = [name for name, count in result]
    assert "Университет А" in university_names
    assert "Университет Б" in university_names

    for university_name, count in result:
        assert isinstance(university_name, str)

```

```

    assert isinstance(count, int)

    assert count == 2

def test_get_faculties_ending_with_iy(self):
    """Тест 3: Факультеты на 'ий' и их университеты"""
    result = get_faculties_ending_with_iy(
        self.faculties,
        self.universities,
        self.faculty_universities
    )

    assert len(result) == 3

    faculty_names = [name for name, unis in result]
    assert "Филологический" in faculty_names
    assert "Математический" in faculty_names
    assert "Экономический" in faculty_names

    for faculty_name, university_list in result:
        if faculty_name == "Филологический":
            assert len(university_list) == 2
            assert "Университет А" in university_list
            assert "Университет Б" in university_list
            break

if __name__ == "__main__":
    tests = TestUniversityQueries()
    tests.setup_method()

```

```

print("Запуск теста 1...")
tests.test_get_university_faculties()
print("✓ Тест 1 пройден")

print("\nЗапуск теста 2...")
tests.test_get_faculty_count()
print("✓ Тест 2 пройден")

print("\nЗапуск теста 3...")
tests.test_get_faculties_ending_with_iy()
print("✓ Тест 3 пройден")

print("\n🟢 Все тесты успешно пройдены!")

```

Результат выполнения программы:

```

(myenv) kaguya80@DESKTOP-BCOUQ5P:~/Kamenskih-CS-Labs-3-sem$ python3 test_queries.py -v
Запуск теста 1...
✓ Тест 1 пройден

Запуск теста 2...
✓ Тест 2 пройден

Запуск теста 3...
✓ Тест 3 пройден

🟢 Все тесты успешно пройдены!

```

```

(myenv) kaguya80@DESKTOP-BCOUQ5P:~/Kamenskih-CS-Labs-3-sem$ python3 simple_tests.py
=====
Запуск модульных тестов
=====

Тест 1: get_university_faculties
Результат: {'Тестовый Университет 1': ['Тестовый факультет'], 'Тестовый Университет 2': ['Другой факультет']}
Тест 1 пройден

Тест 2: get_faculty_count
Результат: [('Тестовый Университет 1', 1), ('Тестовый Университет 2', 1)]
Тест 2 пройден

Тест 3: get_faculties_ending_with_iy
Результат: [('Химический', [])]
Тест 3 пройден

=====
Все тесты успешно пройдены!
=====

```